MakeMyMarrige

API DOCUMENTATION

Version 1.0.0 | 4 November 2026

REST API • NestJS • OpenAPI 3.1 • Member + Guest + Provider Interfaces

Contract version: 1.0.0

Base path: `/v1`

Protocol: HTTPS REST + JSON

Backend: NestJS + TypeScript

Database: MongoDB Atlas

Timezone: `Asia/Kolkata`

Status: API design baseline; implementation and conformance testing required.

1. API design principles

• Resource-oriented REST endpoints; nouns for resources and explicit action subresources for state transitions.

• Version all public routes under `/v1`; breaking changes require a new major API version.

• JSON request/response bodies; UTF-8; ISO 8601 UTC timestamps; `YYYY-MM-DD` for date-only values.

• Internal IDs are opaque strings to clients. Never expose database implementation details as a client contract.

• The server derives tenant context from authenticated membership. `weddingId` in client input never grants access.

• Cookie-based member and guest sessions; no long-lived role claims trusted without current membership/permission validation.

• Use transactions for atomic multi-document business invariants; transactional outbox for side effects.

• Use idempotency keys for retryable create/payment/financial operations; optimistic concurrency for mutable resources.

2. Environments and base URLs

• Production: replace `https://api.example.com` with the deployed API origin.

• Staging: replace `https://staging-api.example.com` with the staging origin.

• Local: `http://localhost:<port>/v1` only for local development; never expose plain HTTP publicly.

3. Authentication and actor contexts

Member sessions

Email/password or Google OAuth establishes an account session. Access and refresh credentials are Secure, HttpOnly cookies. Refresh rotates credentials and detects reuse. Email verification is required before workspace creation and invitation acceptance. Unsafe cookie-authenticated requests require CSRF protection.

Guest sessions

Guests do not create platform accounts. A wedding-scoped OTP challenge verifies control of a phone number and resolves a guest record. A guest session is restricted to one wedding and one guest identity. Shared phone numbers do not merge guest identities.

Webhooks

Payment provider webhooks use provider-specific signature verification, timestamp/replay checks and event deduplication. They do not use member or guest cookies.

4. Tenant resolution and authorization

1. Authenticate actor/session.

2. Resolve active membership or guest session from authoritative state.

3. Resolve weddingId server-side and attach immutable request context.

4. Check role, action-level permission and resource scope.

5. Query through tenant-scoped repositories; every tenant record relationship is checked against the same weddingId.

6. Re-check permission for photo downloads, exports and notification deep links.

Never accept a client-provided weddingId as authority. If present for route/resource addressing, compare it with the resolved tenant.

Role policy summary

• **Co-owner:** full workspace governance and owner-level operations; high-impact actions may require both co-owners.

• **Organiser:** only explicit action-level permissions and scopes. No wedding-wide financial ledger, budget totals or Shagun totals.

• **Treasurer:** dedicated finance role; normal expense approval and payment recording. Treasurer-created or Treasurer-beneficiary expenses route to both co-owners.

• **Guest:** own profile/session, visible published function data, own RSVP, own additional-guest requests, permitted gallery actions and own Shagun history.

• **Platform support:** break-glass, restricted, case-bound and fully audited; no routine workspace administration.

5. Request conventions

Headers

```
Accept: application/json
Content-Type: application/json
X-Request-Id: <optional-client-correlation-id>
Idempotency-Key: <opaque-key-for-retryable-mutation>
X-CSRF-Token: <required-when-cookie-CSRF-policy-demands>
If-Match: <resource-version-or-etag>
```

`X-Request-Id` is accepted only as a correlation hint; the server validates or replaces it. `Idempotency-Key` is required for payment-intent creation and recommended for invitation acceptance, expense payment recording, export requests and other retryable creates. `If-Match` may be used where the API exposes ETags; otherwise use `expectedVersion` in the request body.

Pagination

• Cursor pagination for large collections; default limit 20, maximum 50.

• Sort is server-allowlisted. Stable default order is `createdAt DESC, _id DESC`.

• Response: `meta.nextCursor`, `meta.hasMore`, `meta.limit`.

• Cursors are opaque, signed or integrity-protected and bound to tenant, filters and sort.

• Avoid deep `skip` pagination and unrestricted client-selected sort fields.

Filtering and search

• Only documented filters are accepted; validate enum values, date ranges and maximum query lengths.

• Guest search must be tenant-scoped and rate-limited. Do not expose unrestricted regex search.

• Vendor discovery is mediated through the provider adapter; do not expose provider API keys.

Dates and money

• Date-only values: `YYYY-MM-DD`, interpreted in `Asia/Kolkata` for wedding scheduling.

• Instants: RFC 3339/ISO 8601 with timezone; responses normalised to UTC (`Z`).

• Monetary request values: integer `amountPaise`, currency `INR`; never accept floating rupees.

• When presenting rupees, round up to the next whole rupee: `ceil(amountPaise / 100)`. This is display-only; stored and settled amounts remain exact paise.

6. Response envelope

Successful responses use a stable envelope:

```
{
  "data": { "id": "opaque-resource-id" },
  "meta": { "requestId": "req_...", "servedAt": "2026-11-04T10:00:00Z" }
}
```

List responses put an array in `data` and cursor metadata in `meta`.

7. Error model

```
{
  "error": {
    "code": "ACTIVE_WEDDING_MEMBERSHIP_EXISTS",
    "message": "This account already has an active wedding membership.",
    "requestId": "req_...",
    "details": [],
    "retryable": false
  }
}
```

Stable machine-readable `code` is the contract. Human-readable messages may evolve. Do not return stack traces, raw database errors, token state or another tenant's private details.

HTTP status policy

| Status | Meaning |

|---|---|

| 200 | Successful read/update/action |

| 201 | Resource created |

| 202 | Accepted for asynchronous processing |

| 204 | Successful action with no response body (use sparingly) |

| 400 | Malformed request or invalid syntax |

| 401 | Missing/invalid authentication |

| 403 | Authenticated but not authorised |

| 404 | Resource absent or intentionally concealed |

| 409 | State conflict, duplicate, stale version, membership conflict |

| 410 | Expired/revoked invitation or token |

| 412 | If-Match precondition failed |

| 422 | Syntactically valid but domain validation failed |

| 429 | Rate limited; include Retry-After |

| 500 | Unexpected server failure |

| 503 | Temporarily unavailable/dependency circuit open |

8. Concurrency, idempotency and state transitions

• Mutable resources expose `version`; update with expected version or If-Match. Stale writes return 409 (or 412 for If-Match).

• State transitions are explicit action endpoints, not arbitrary status patching.

• The API validates allowed transition, actor permission, required approvals and current resource version.

• Idempotency records bind actor + operation scope + key + request hash. Same key/same payload returns original outcome; same key/different payload returns 409.

• Webhook handlers deduplicate provider event IDs and tolerate duplicate/out-of-order delivery.

• BullMQ consumers are retry-safe and idempotent. Queue delivery is not exactly-once.

9. Rate limiting

• Cloudflare edge layer: WAF, DDoS and coarse IP controls.

• NestJS/Redis layer: account, session, guest, phone, wedding, IP and route-aware controls.

• OTP, login, registration, password reset, invitation preview/acceptance, upload-intent and payment-intent routes receive stricter limits.

• Return 429 with `Retry-After`; do not reveal whether a phone/email belongs to an account.

• Redis outage: fail closed for auth/OTP/payment/upload-sensitive operations; conservative per-instance emergency limits for ordinary reads.

10. Security and privacy requirements

• HTTPS only in deployed environments; HSTS at edge; secure cookie flags.

• CSRF protection for unsafe cookie-authenticated requests; CORS allowlist exact frontend origins.

• DTO validation and output allowlisting; reject mass assignment and unknown fields.

• Never log passwords, OTPs, refresh tokens, invitation tokens, payment credentials or raw sensitive provider payloads.

• Store only token hashes; rotate refresh tokens; revoke token family on reuse detection.

• Rate limit enumeration-sensitive endpoints and use generic responses.

• Private R2 bucket; presigned uploads/downloads are short-lived; authorise every download.

• Remove GPS EXIF before publication; validate MIME by decoding actual file bytes, not filename.

• Payment webhook signature, timestamp and replay validation are mandatory.

• Audit sensitive governance, finance, guest, vendor, photo and export actions.

11. Workflow API behaviour

• **WF01:** account registration/login, email verification, workspace creation, co-owner invitation and transactional acceptance. Membership conflict returns 409 without consuming invitation.

• **WF02:** dual approvals for organiser invitation/removal and Treasurer changes; immutable proposal snapshot; approval changes invalidate when proposal version changes.

• **WF03:** function draft edits; publication/cancellation dual-approved; guest-facing reads use immutable published snapshots. Task completion checks mandatory checklist and dependencies.

• **WF04:** household and guest records; corrections reviewed by someone other than requester; merge dual-approved; shared phone numbers allowed.

• **WF05:** guest self-RSVP until deadline; one record per wedding/function/guest; assisted corrections require permission and reason; unanswered remains pending.

• **WF06:** paise-only finance; normal expense Treasurer approval; Treasurer-created/beneficiary expense requires both co-owners; only Treasurer records payments; void/correction follows dual approval.

• **WF07:** Google Places adapter and manual vendors; explicit availability state; selection is not booking; booking status recorded separately.

• **WF08:** one approved UPI destination; provider-verified webhook only marks contribution successful; manual reconciliation cannot mark success.

• **WF09:** direct-to-R2 presigned upload; quarantine and validation; guest uploads moderated; signed download URLs short-lived; all reads authorised.

• **WF10:** in-app notifications only; dashboard visibility by role; financial summaries restricted to co-owners and Treasurer.

• **WF11:** audit visibility co-owner only; sensitive exports audited; support recovery case-bound and restricted.

• **WF12:** cancellation second approval revokes access immediately; seven-day public notice only; deletion runs asynchronously and idempotently.

12. Endpoint catalogue

Audit

• `GET /v1/audit-events` — List audit events (co-owners only).

• `POST /v1/audit-exports` — Request audit CSV/PDF export.

• `GET /v1/audit-exports/{exportId}` — Get export status and authorised short-lived URL.

Authentication

• `POST /v1/auth/register` — Register account.

• `POST /v1/auth/login` — Authenticate with email and password.

• `POST /v1/auth/google` — Exchange Google OAuth credential/code for an application session.

• `POST /v1/auth/refresh` — Rotate refresh session and detect token reuse.

• `POST /v1/auth/logout` — Revoke current session.

• `GET /v1/auth/me` — Get current account and active membership.

• `GET /v1/auth/sessions` — List account sessions.

• `POST /v1/auth/sessions/{sessionId}/revoke` — Revoke one account session.

• `POST /v1/auth/email/verify` — Consume email verification token.

• `POST /v1/auth/email/resend` — Resend verification email.

• `POST /v1/auth/password/forgot` — Request password reset.

• `POST /v1/auth/password/reset` — Consume reset token and set new password.

Dashboard

• `GET /v1/dashboard` — Get role-specific dashboard metrics.

• `GET /v1/dashboard/alerts` — Get authorised dashboard alerts.

Digital Shagun

• `GET /v1/shagun/settings` — Get masked Shagun configuration.

• `POST /v1/shagun/configuration-requests` — Propose UPI destination activation/change.

• `POST /v1/shagun/configuration-requests/{requestId}/approve` — Approve destination configuration as co-owner.

• `POST /v1/shagun/payment-intents` — Create provider payment intent for authenticated guest.

• `GET /v1/shagun/contributions` — List contributions for authorised finance roles.

• `GET /v1/shagun/contributions/mine` — List own verified contribution history.

• `POST /v1/shagun/payment-intents/{intentId}/reconcile` — Initiate provider status check; cannot manually mark paid.

Finance

• `GET /v1/finance/budget` — Get budget and category allocation summary.

• `PATCH /v1/finance/budget` — Update budget only through governance policy.

• `GET /v1/finance/categories` — List budget categories.

• `POST /v1/finance/categories` — Create budget category.

• `PATCH /v1/finance/categories/{categoryId}` — Update category through authorised governance.

• `GET /v1/finance/estimates` — List expense estimates.

• `POST /v1/finance/estimates` — Create estimate range.

• `GET /v1/finance/expenses` — List expenses visible to caller.

• `POST /v1/finance/expenses` — Create expense.

• `GET /v1/finance/expenses/{expenseId}` — Get expense detail.

• `PATCH /v1/finance/expenses/{expenseId}` — Update draft expense only.

• `POST /v1/finance/expenses/{expenseId}/submit` — Submit expense for approval.

• `POST /v1/finance/expenses/{expenseId}/approve` — Approve using Treasurer or co-owner exception route.

- `POST /v1/finance/expenses/{expenseId}/reject` — Reject expense.
- `POST /v1/finance/expenses/{expenseId}/payments` — Record payment entry (Treasurer only).
- `GET /v1/finance/expenses/{expenseId}/payments` — List expense payment entries.
- `POST /v1/finance/expenses/{expenseId}/void-requests` — Request void of approved expense.
- `POST /v1/finance/expenses/{expenseId}/adjustments` — Request correction/reversal adjustment.
- `POST /v1/finance/reserve-transfers` — Propose reserve allocation transfer with threshold governance.

Functions

- `GET /v1/functions` — List functions.
- `POST /v1/functions` — Create function draft.
- `GET /v1/functions/{functionId}` — Get function detail.
- `PATCH /v1/functions/{functionId}` — Update unpublished function draft.
- `POST /v1/functions/{functionId}/publish` — Submit or execute dual-approved publication.
- `POST /v1/functions/{functionId}/cancel` — Cancel function using dual approval.
- `GET /v1/functions/{functionId}/publication-history` — List immutable publication snapshots.

Governance

- `GET /v1/workspaces/current/governance-requests` — List governance proposals.
- `POST /v1/workspaces/current/governance-requests` — Create governance proposal.
- `GET /v1/workspaces/current/governance-requests/{requestId}` — Get governance proposal.
- `POST /v1/workspaces/current/governance-requests/{requestId}/approve` — Approve proposal as distinct active co-owner.
- `POST /v1/workspaces/current/governance-requests/{requestId}/reject` — Reject proposal.
- `POST /v1/workspaces/current/governance-requests/{requestId}/cancel` — Cancel proposal before execution.
- `GET /v1/workspaces/current/finance-settings` — Get Treasurer and finance governance settings.

Guest Authentication

- `POST /v1/guest-auth/otp/request` — Request guest OTP.
- `POST /v1/guest-auth/otp/verify` — Verify OTP and issue wedding-scoped guest session.
- `POST /v1/guest-auth/logout` — Revoke guest session.

Guest Portal

- `GET /v1/guest/me` — Get authenticated guest's own profile and permitted wedding context.
- `GET /v1/guest/functions` — List functions for which guest has visibility/invitation.
- `GET /v1/guest/functions/{functionId}` — Get published function snapshot visible to guest.
- `GET /v1/guest/rsvps` — List own RSVP records.
- `POST /v1/guest/functions/{functionId}/rsvp` — Create or update own RSVP before deadline.
- `POST /v1/guest/functions/{functionId}/additional-guest-requests` — Request additional guests.
- `GET /v1/guest/additional-guest-requests` — List own additional guest requests.

Guests

- `GET /v1/households` — List households.
- `POST /v1/households` — Create household.
- `GET /v1/households/{householdId}` — Get household and permitted guest summaries.
- `PATCH /v1/households/{householdId}` — Update household.
- `POST /v1/households/{householdId}/archive` — Archive household without deleting guests.
- `GET /v1/guests` — Search/list guests with tenant-scoped filters.
- `POST /v1/guests` — Create guest.
- `GET /v1/guests/{guestId}` — Get guest profile.
- `PATCH /v1/guests/{guestId}` — Update allowed guest fields.
- `POST /v1/guests/{guestId}/correction-requests` — Submit profile correction request.
- `POST /v1/guests/{guestId}/merge-requests` — Propose guest merge for dual approval.
- `POST /v1/guests/{guestId}/household-transfer` — Transfer guest between households preserving history.
- `GET /v1/relationship-tags` — List wedding relationship tags.
- `POST /v1/relationship-tags` — Create relationship tag.

Invitations

• `POST /v1/invitations/preview` — Preview invitation with minimal pre-auth disclosure.

• `POST /v1/invitations/{invitationId}/accept` — Accept invitation transactionally.

• `GET /v1/workspaces/current/invitations` — List workspace invitations.

• `POST /v1/workspaces/current/invitations` — Create invitation after eligibility/governance checks.

• `POST /v1/workspaces/current/invitations/{invitationId}/revoke` — Revoke pending invitation.

Lifecycle

• `POST /v1/workspaces/current/lifecycle/cancel-requests` — Request workspace cancellation requiring two co-owner approvals.

• `POST /v1/workspaces/current/lifecycle/cancel-requests/{requestId}/approve` — Second co-owner approval triggers immediate access revocation and deletion workflow.

• `POST /v1/workspaces/current/lifecycle/date-change-requests` — Request wedding date change under post-date guardrails.

• `POST /v1/workspaces/current/exports` — Request final workspace export before cancellation approval completes.

• `GET /v1/workspaces/current/deletion-status` — Get deletion progress where caller remains authorised.

Membership

• `GET /v1/workspaces/current/members` — List workspace members.

• `GET /v1/workspaces/current/members/{membershipId}` — Get member details.

• `POST /v1/workspaces/current/members/{membershipId}/suspend` — Emergency suspend non-co-owner member.

• `POST /v1/workspaces/current/members/{membershipId}/governance-removal` — Propose member removal through dual approval.

Notifications

• `GET /v1/notifications` — List own in-app notifications.

• `POST /v1/notifications/{notificationId}/read` — Mark notification read.

• `POST /v1/notifications/read-all` — Mark all visible notifications read.

• `GET /v1/notification-preferences` — Get notification preferences.

• `PATCH /v1/notification-preferences` — Update category preferences.

Photos

• `GET /v1/photos/albums` — List function albums.

• `GET /v1/photos/albums/{albumId}/photos` — List authorised approved photos.

• `POST /v1/photos/upload-intents` — Create upload reservation and presigned R2 URL.

• `POST /v1/photos/uploads/{uploadId}/complete` — Verify object and enqueue validation/moderation.

• `GET /v1/photos/{photoId}` — Get authorised photo metadata and short-lived download URL.

• `POST /v1/photos/{photoId}/reports` — Report published photo.

• `POST /v1/photos/{photoId}/removal-requests` — Request photo removal.

• `POST /v1/photos/{photoId}/moderate` — Approve/reject/hide/restore photo with permission check.

• `POST /v1/photos/{photoId}/permanent-removal` — Co-owner permanent removal decision.

RSVP Management

• `GET /v1/rsvps` — List RSVP records for authorised members.

• `PATCH /v1/rsvps/{rsvpId}/assisted-update` — Assisted RSVP correction with explicit permission and reason.

• `GET /v1/additional-guest-requests` — List additional guest requests.

• `POST /v1/additional-guest-requests/{requestId}/arrangements` — Record partial approval/arrangements.

Tasks

• `GET /v1/tasks` — List tasks.

• `POST /v1/tasks` — Create task.

• `GET /v1/tasks/{taskId}` — Get task detail.

• `PATCH /v1/tasks/{taskId}` — Update task.

• `POST /v1/tasks/{taskId}/assignments` — Assign member(s).

• `POST /v1/tasks/{taskId}/checklist-items` — Add checklist item.

• `PATCH /v1/tasks/{taskId}/checklist-items/{itemId}` — Update checklist item.

• `POST /v1/tasks/{taskId}/complete` — Complete task after mandatory checklist/dependency validation.

• `POST /v1/tasks/{taskId}/reopen` — Reopen task with reason and dependency review.

• `POST /v1/tasks/{taskId}/comments` — Add task comment.

Vendors

• `GET /v1/vendor-categories` — List wedding vendor categories.

• `POST /v1/vendor-categories` — Create vendor category.

• `GET /v1/vendors` — List wedding vendor shortlist.

• `POST /v1/vendors` — Add vendor from provider reference or manual record.

• `GET /v1/vendors/{vendorId}` — Get vendor record.

• `PATCH /v1/vendors/{vendorId}` — Update permitted vendor data.

• `POST /v1/vendors/discovery/search` — Search Google Places through provider adapter.

• `POST /v1/vendors/{vendorId}/enquiries` — Create/update enquiry state.

• `POST /v1/vendors/{vendorId}/quotes` — Record vendor quote.

• `POST /v1/vendors/{vendorId}/availability-checks` — Record explicit availability check.

• `POST /v1/vendors/{vendorId}/select` — Select vendor (co-owner only).

• `POST /v1/vendors/{vendorId}/booking` — Record booking lifecycle explicitly.

Webhooks

• `POST /v1/webhooks/payments/{providerId}` — Receive signed payment-provider event.

Workspace

• `POST /v1/workspaces` — Create wedding workspace and first co-owner membership.

• `GET /v1/workspaces/current` — Get current wedding workspace.

• `PATCH /v1/workspaces/current` — Update permitted workspace profile.

• `GET /v1/workspaces/current/setup-progress` — Get progressive setup checklist.

13. Critical endpoint contract examples

Accept an invitation

```
POST /v1/invitations/{invitationId}/accept
Idempotency-Key: <key>
{ "token": "<opaque-invitation-token>" }
```

Rules: authenticated verified email must exactly match invited email; transaction checks active/suspended membership slot; if occupied, return 409 `ACTIVE_WEDDING_MEMBERSHIP_EXISTS` and leave invitation pending; successful acceptance creates/reactivates membership and records audit/outbox.

Guest RSVP

```
POST /v1/guest/functions/{functionId}/rsvp
{ "response": "ATTENDING", "expectedVersion": 2 }
```

Rules: guest session must resolve to this guest and wedding; guest must be individually invited; deadline must be open; upsert uses unique `(weddingId,functionId,guestId)`; append RSVP event; never accept a guestId from request body.

Create expense

```
POST /v1/finance/expenses
Idempotency-Key: <key>
{ "categoryId": "...", "title": "Venue advance", "beneficiary": "vendor-or-member-reference", "amountPaise": 2500000, "cu
rrency": "INR" }
```

Rules: validate category and beneficiary in same wedding; creator cannot self-approve; submit/approve are separate actions; approval route is Treasurer or both co-owners for Treasurer-related expense.

Create photo upload intent

```
POST /v1/photos/upload-intents
{ "functionId": "...", "fileName": "photo.jpg", "mimeType": "image/jpeg", "byteSize": 2400000 }
```

Response includes a short-lived presigned PUT URL, required headers, uploadId and expiry. Client uploads directly to private R2; then calls completion endpoint. API verifies object metadata and queues byte-level validation, EXIF stripping and moderation.

Payment webhook

```
POST /v1/webhooks/payments/{providerId}
```

Rules: validate provider signature and timestamp; persist/deduplicate event; acknowledge quickly; process asynchronously; reconcile amount, currency, intent and beneficiary; only provider-verified final state creates verified contribution.

14. Webhook and asynchronous processing

• Webhook ingress validates signature/replay window and persists inbox event before acknowledging.

• Outbox event is committed in same transaction as domain mutation.

• Dispatcher publishes to BullMQ; consumers use inbox/idempotency keys and retry-safe state transitions.

• External API calls occur outside MongoDB transactions.

• Dead-letter/failed events are observable and manually replayable through restricted operational tooling.

15. OpenAPI / Swagger and Postman usage

• Import `openapi.yaml` or `openapi.json` into Swagger UI, Redoc or Postman.

• Replace placeholder server URLs with deployed origins.

• Configure cookie auth/CSRF in the API test environment; browser cookie auth may not be fully testable from every API client.

• Treat schemas and endpoint descriptions as a contract baseline. Run OpenAPI linting and contract tests in CI.

• Do not publish internal admin/support endpoints or real provider webhook secrets in public API docs.

16. Product decisions deliberately standardised

• REST over GraphQL for pilot: aligns with workflow/state-transition APIs, predictable caching and OpenAPI tooling.

• `/v1` version prefix; JSON envelope; cursor pagination; stable error codes.

• Secure HttpOnly cookie sessions for browser clients; guest cookie sessions separate.

• Action endpoints for irreversible/high-governance transitions; no arbitrary status PATCH.

• Member API and guest API are separate security surfaces.

• No vendor account API, public photo API, WhatsApp notification API, or guest platform account API in v1.

17. Release checklist

• Every endpoint has an owner, permission requirement, DTO, response schema and error mapping.

• Cross-tenant access tests for every tenant resource and worker.

• Auth/session rotation, CSRF, rate-limit and account-enumeration tests.

• Idempotency and concurrent request tests for invitation, finance and payment operations.

• Provider webhook signature/replay tests and reconciliation tests.

• Photo upload content validation, EXIF removal, access and signed URL expiry tests.

• OpenAPI lint, generated client compatibility and backward-compatibility checks.

• Production URLs, CORS origins, cookie domain, rate policies, request size limits and observability configured.

18. Explicit unresolved implementation inputs

• Production API hostname and frontend origin (deployment decisions, not required to design the contract).

• Final OTP provider and whether OTP challenge storage is MongoDB-only or provider-assisted.

• Final UPI provider and exact webhook signature/settlement contract.

• Exact organiser permission catalogue and scope grammar, to be implemented from approved permission matrix.

• Final upload byte limits, allowed image dimensions and per-wedding storage quotas, to be set from hosting/cost limits.

• SLOs and numeric rate-limit thresholds, to be established from pilot measurements and load tests.